Neural Networks Regression Business Case

By Atharva Nitin Deshpande (Scaler DSML August 2024 Batch)

Introduction

Problem Statement

Porter, India's largest marketplace for intra-city logistics, is revolutionizing the delivery sector with technology-driven solutions. This case focuses on leveraging neural networks to accurately predict delivery times, a critical aspect of customer satisfaction in logistics. With a dataset encompassing various aspects of orders and deliveries, Porter aims to refine its delivery time estimations. Analyzing this dataset can provide significant insights into delivery dynamics, efficiency bottlenecks, and optimization opportunities. The insights obtained can enhance Porter's operational efficiency, ensuring timely deliveries and improving driver-partner allocation.

Let us now begin this business case by importing important libraries.

In [127…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import seaborn as sns

sns.set_style('darkgrid')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
from tensorflow.keras.utils import plot_model
import pydot
from sklearn.metrics import r2_score
import joblib
```

Let us now import the raw data.

In [128…

```
raw_data = pd.read_csv('data_2.csv')
df = raw_data.copy(deep=True)
```

In [129…

```
df.head()
```

Out[129…

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_dashers	total_busy_dashers	total_outstan
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	1.0	4	3441	4	557	1239	33.0	14.0	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	2.0	1	1900	1	1400	1400	1.0	2.0	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	3.0	4	4771	3	820	1604	8.0	6.0	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	1.0	1	1525	1	1525	1525	5.0	6.0	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	1.0	2	3620	2	1425	2195	5.0	5.0	

In [130…

```
df.tail()
```

Out[130…

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_dashers	total_busy_dashers	total_ou
175772	1.0	2015-02-17 00:19:41	2015-02-17 01:02:41	28	4.0	3	1389	3	345	649	17.0	17.0	
175773	1.0	2015-02-13 00:01:59	2015-02-13 01:03:59	28	4.0	6	3010	4	405	825	12.0	11.0	
175774	1.0	2015-01-24 04:46:08	2015-01-24 05:32:08	28	4.0	5	1836	3	300	399	39.0	41.0	
175775	1.0	2015-02-01 18:18:15	2015-02-01 19:03:15	58	1.0	1	1175	1	535	535	7.0	7.0	
175776	1.0	2015-02-08 19:24:33	2015-02-08 20:01:33	58	1.0	4	2605	4	425	750	20.0	20.0	

In [131…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  object
 2   actual_delivery_time                          175777 non-null  object
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: float64(6), int64(6), object(2)
memory usage: 18.8+ MB
```

Each row in this data corresponds to one unique delivery. Each column corresponds to a feature as explained below.

`market_id` : integer id for the market where the restaurant lies

`created_at` : the timestamp at which the order was placed

`actual_delivery_time` : the timestamp when the order was delivered

`store_primary_category` : category for the restaurant

`order_protocol` : integer code value for order protocol(how the order was placed ie: through porter, call to restaurant, pre booked, third part etc)

`total_items subtotal` : final price of the order

`num_distinct_items` : the number of distinct items in the order

`min_item_price` : price of the cheapest item in the order

`max_item_price` : price of the costliest item in order

`total_onshift_partners` : number of delivery partners on duty at the time order was placed

`total_busy_partners` : number of delivery partners attending to other tasks

`total_outstanding_orders` : total number of orders to be fulfilled at the moment

`estimated_store_to_consumer_driving_duration` : approximate travel time from restaurant to customer

In [ ]:
```python
# Number of unique values
for col in df.columns:
    print(f'{col}: {df[col].nunique()}')
```

```
market_id: 6
created_at: 162649
actual_delivery_time: 160344
store_primary_category: 73
order_protocol: 7
total_items: 54
subtotal: 8182
num_distinct_items: 20
min_item_price: 2251
max_item_price: 2585
total_onshift_dashers: 172
total_busy_dashers: 158
total_outstanding_orders: 281
estimated_store_to_consumer_driving_duration: 1318
```

In [134…

```python
# Inspect market_id, store_primary_category and order_protocol
for col in ['market_id', 'store_primary_category', 'order_protocol']:
    print(f'{col}\n{df[col].unique()}\n')
```

```
market_id
[1. 2. 3. 4. 5. 6.]

store_primary_category
[ 4 46 36 38 58 68 15 57 55 20 60 13 45 39 34 18 29 23 50 70  6  7 10 28
 24 61 59 72 16 65 47 53 49 40 66 42 21 14 35 11 52  2 71 25 54 69 26 12
 32 30 17 33 22 31  0 51 48 67 44 62  9 27 19  1  5 41 37 64 56 63 43  8
  3]

order_protocol
[1. 2. 3. 5. 4. 6. 7.]
```

In [135…

```python
# Convert market_id, store_primary_category and order_protocol to category
for col in ['market_id', 'store_primary_category', 'order_protocol']:
    df[col] = df[col].astype(int).astype('category')
```

In [136…

```python
df.describe().T
```

Out[136…

	count	mean	std	min	25%	50%	75%	max
total_items	175777.0	3.204976	2.674055	1.0	2.0	3.0	4.0	411.0
subtotal	175777.0	2697.111147	1828.554893	0.0	1412.0	2224.0	3410.0	26800.0
num_distinct_items	175777.0	2.675060	1.625681	1.0	1.0	2.0	3.0	20.0
min_item_price	175777.0	684.965433	519.882924	-86.0	299.0	595.0	942.0	14700.0
max_item_price	175777.0	1160.158616	560.828571	0.0	799.0	1095.0	1395.0	14700.0
total_onshift_dashers	175777.0	44.918664	34.544724	-4.0	17.0	37.0	66.0	171.0
total_busy_dashers	175777.0	41.861381	32.168505	-5.0	15.0	35.0	63.0	154.0
total_outstanding_orders	175777.0	58.230115	52.731043	-6.0	17.0	41.0	85.0	285.0
estimated_store_to_consumer_driving_duration	175777.0	546.077240	218.717798	0.0	384.0	544.0	703.0	2088.0

In [137…
```
df.describe(include=['object','category']).T
```

Out[137…

	count	unique	top	freq
market_id	175777	6	2	53469
created_at	175777	162649	2015-02-11 19:50:43	6
actual_delivery_time	175777	160344	2015-02-14 02:58:40	5
store_primary_category	175777	73	4	18183
order_protocol	175777	7	1	48404

In [ ]:
```
# Check for duplicate rows
df.duplicated().sum()
```

Out[ ]:
```
np.int64(0)
```

Columns `created_at` and `actual_delivery_time` are of the datetime format. The target variable for this study, `estimated_delivery_time`, is to be derived from the difference between them.

In [139…
```
# FEATURE ENGINEERING

# Let us convert created_at and actual_delivery_time into valid datetime format
df['created_at'] = pd.to_datetime(df['created_at'])
df['actual_delivery_time'] = pd.to_datetime(df['actual_delivery_time'])

# Getting hour of day from the order time and also the day of the week
df['hour_of_day'] = df['created_at'].dt.hour
df['day_of_week'] = df['created_at'].dt.day_of_week

# The target variable for this study, 'estimated_delivery_time', is to be derived from the difference between 'created_at' and 'actual_delivery_time'.
df['estimated_delivery_time'] = (df['actual_delivery_time'] - df['created_at']).dt.seconds/60
```

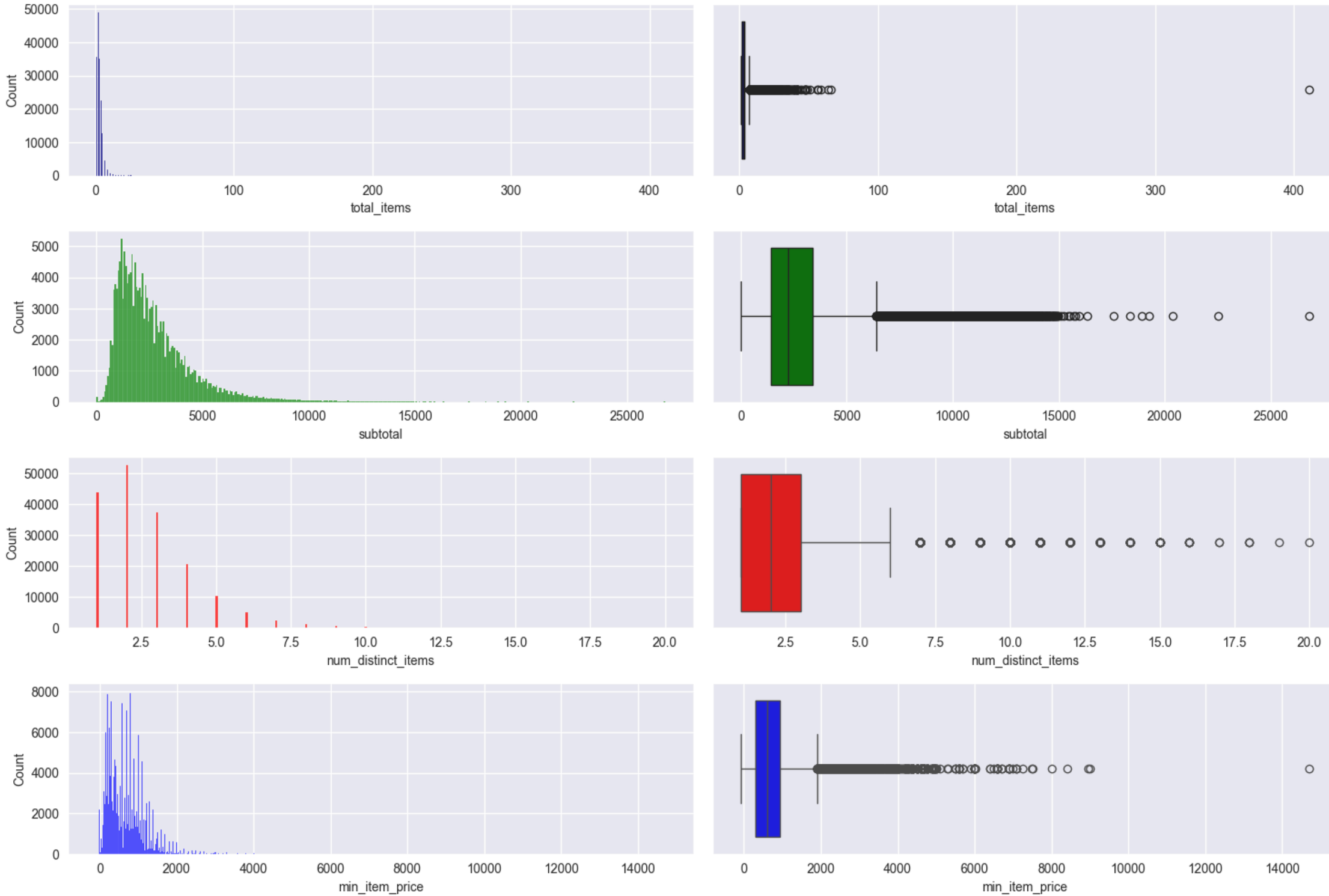
In [140…
```
# We can drop the datetime columns now
df.drop(columns=['created_at','actual_delivery_time'], inplace=True)
```

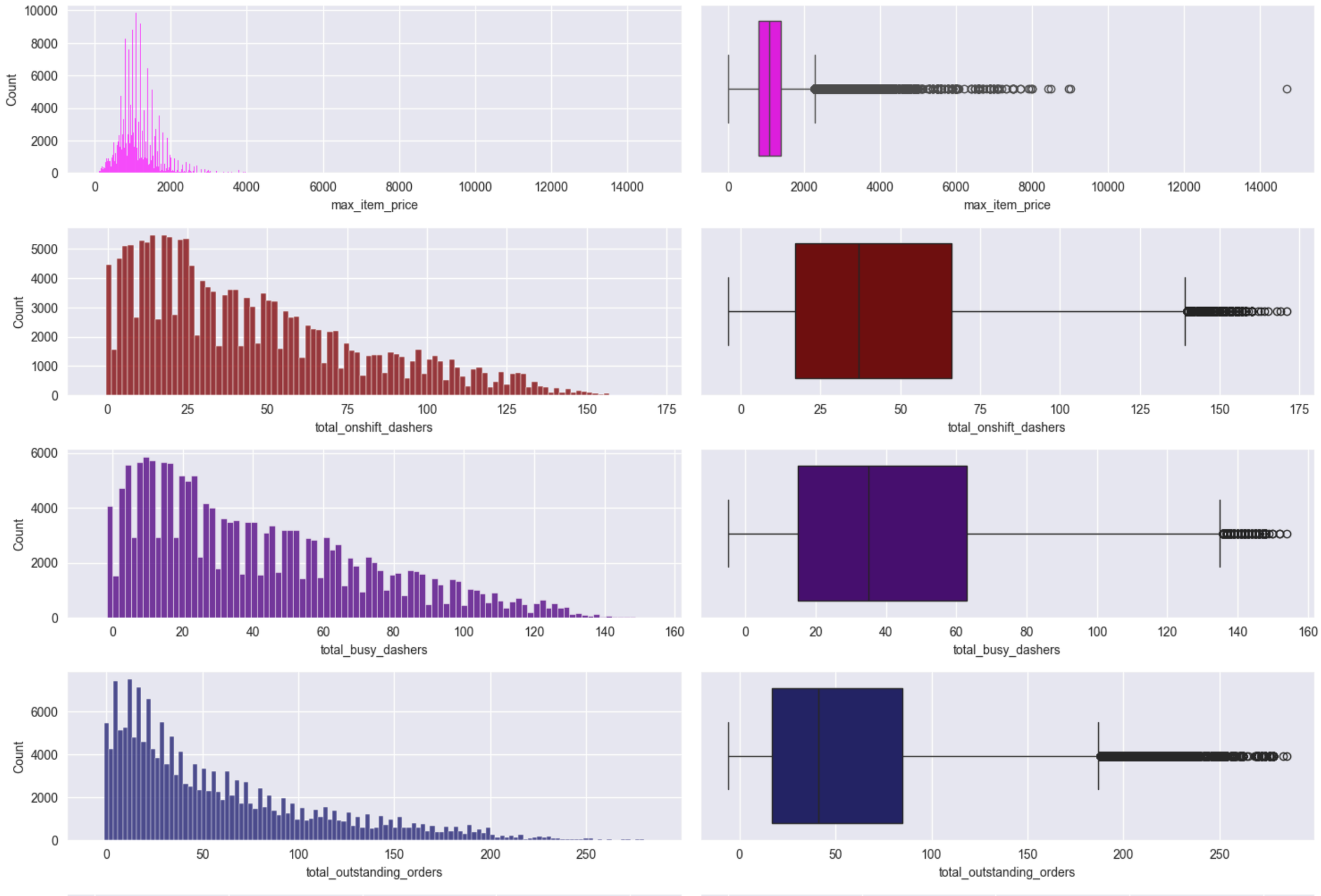
EDA - Univariate Analysis

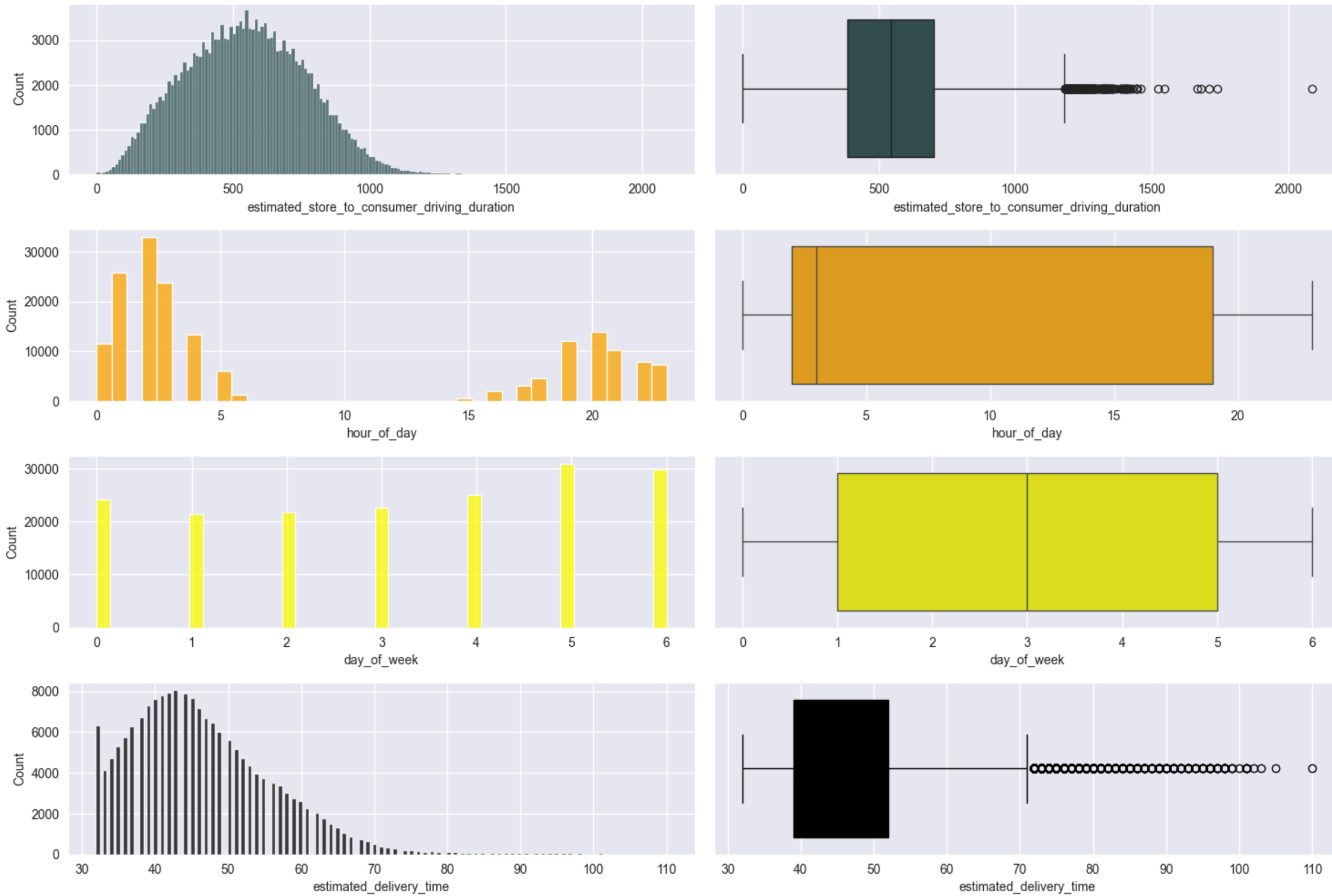
Numerical Features

In [141…
```
num_cols = df.select_dtypes(include='number')
grid_size = num_cols.shape[1]
colors = ['navy', 'green', 'red', 'blue', 'magenta', 'maroon', 'indigo', 'midnightblue', 'darkslategray', 'orange', 'yellow', 'black']
plt.figure(figsize=(15,30))
for i, col in enumerate(num_cols):
    plt.subplot(grid_size, 2, 2*i+1)
    sns.histplot(df[col], color=colors[i])
    plt.subplot(grid_size, 2, 2*i+2)
    sns.boxplot(x=df[col], color=colors[i])
```

```
plt.tight_layout()
plt.show()
```


Count
total_items
subtotal
num_distinct_items
min_item_price

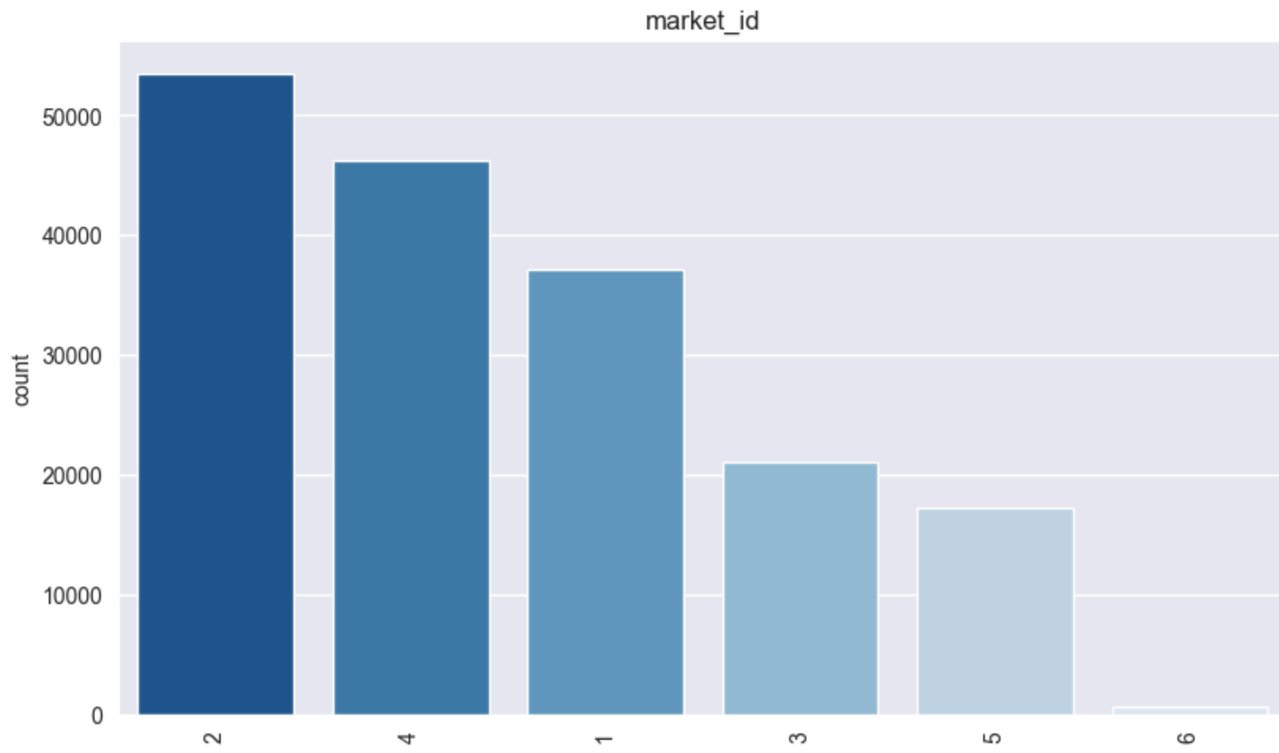

Count
max_item_price
total_onshift_dashers
total_busy_dashers
total_outstanding_orders

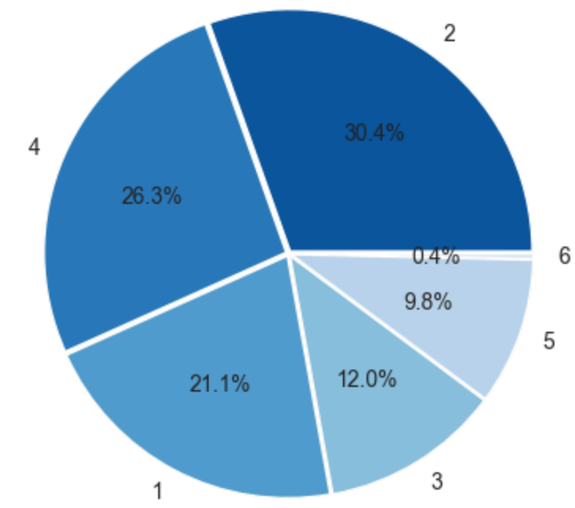

Count
estimated_store_to_consumer_driving_duration
hour_of_day
day_of_week
estimated_delivery_time

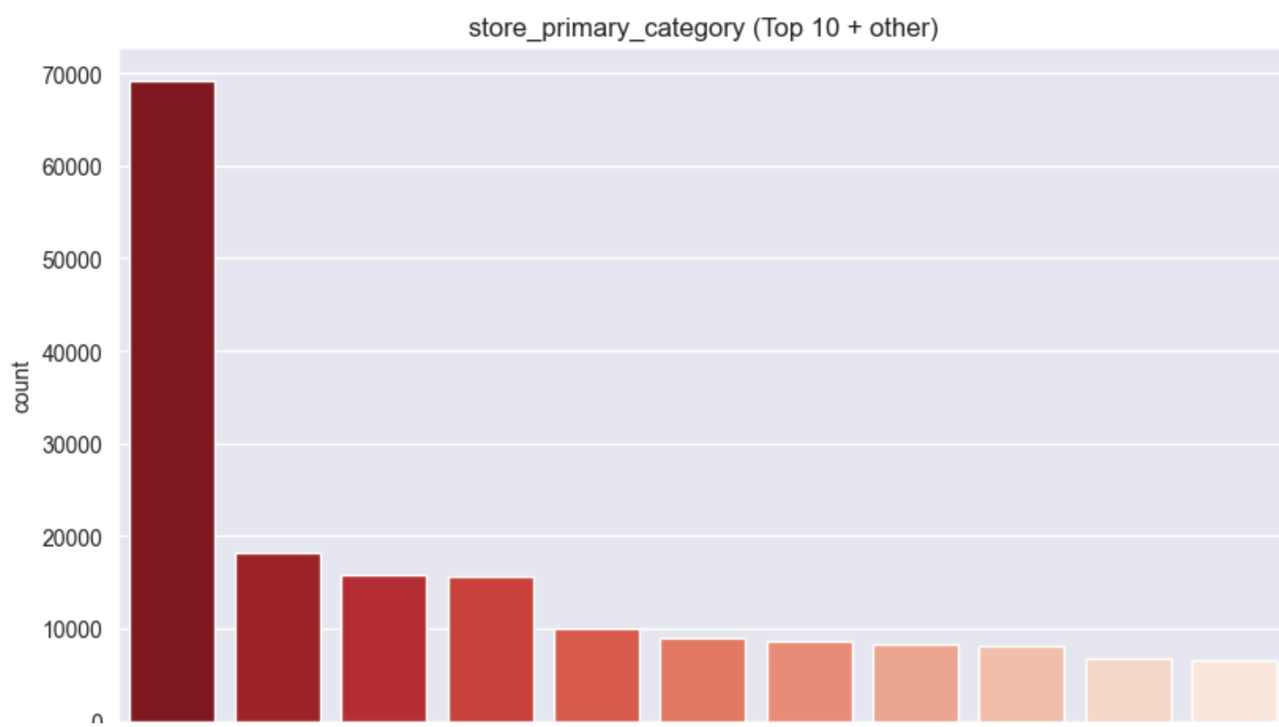
In [ ]:
```
# Capping Upper Outliers at 99.99th percentile values
for col in num_cols:
    df[col] = df[col].clip(upper=df[col].quantile(0.9999))
```

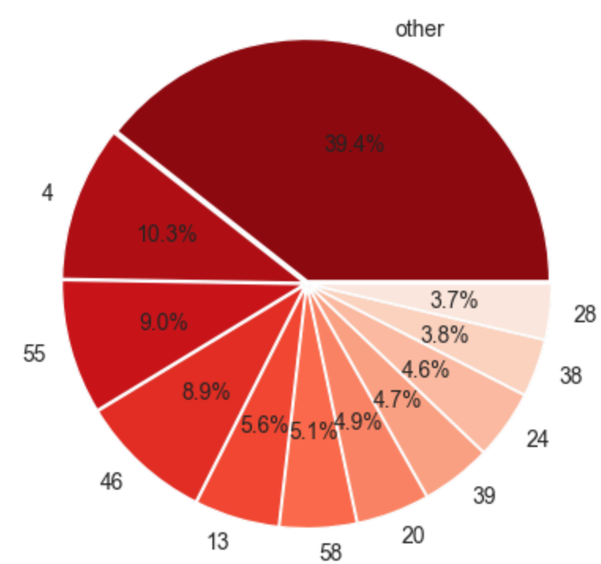
Categorical Features

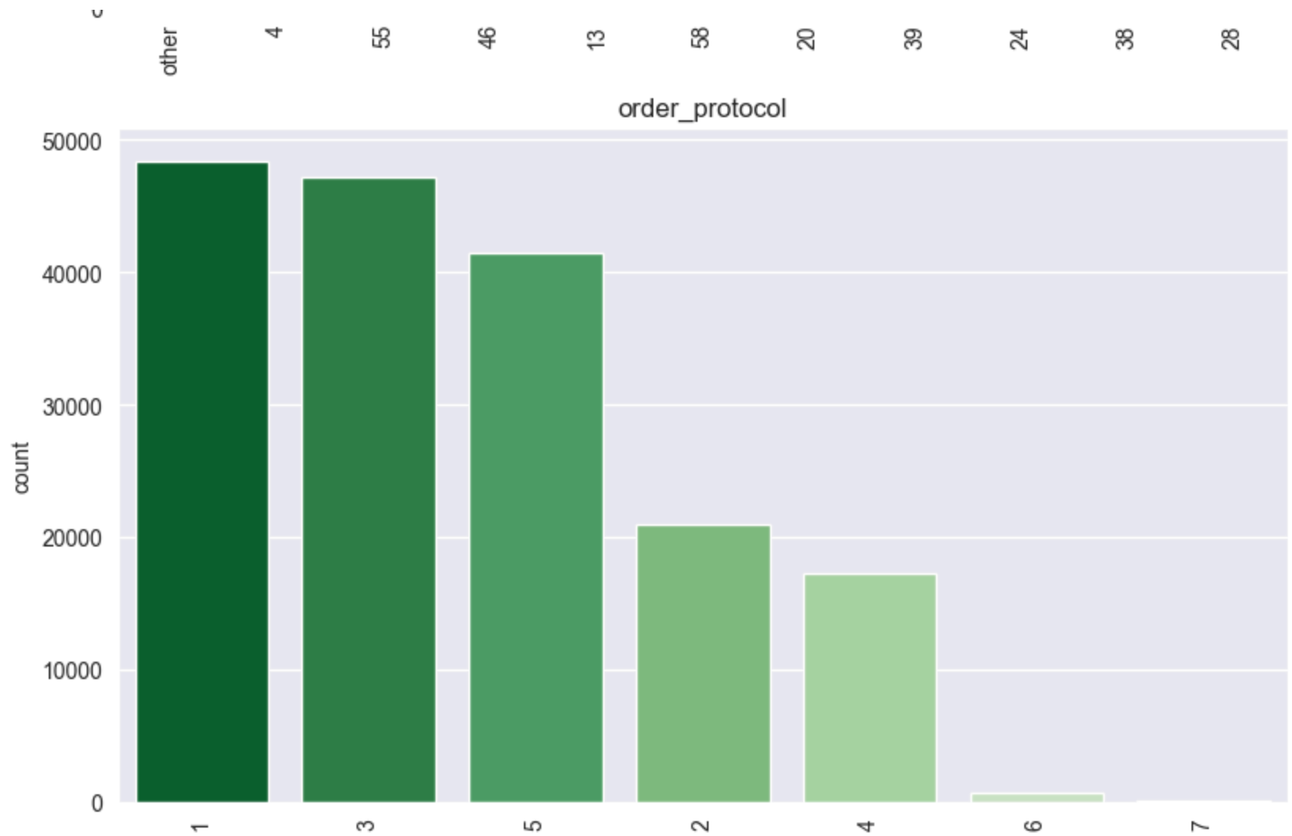
In [142…
```
cat_cols = df.select_dtypes(exclude='number')
grid_size = cat_cols.shape[1]
palettes = ['Blues_r','Reds_r','Greens_r']
plt.figure(figsize=(15,15))
for i, col in enumerate(cat_cols):
    s = df[col]
    if s.nunique() > 10:
        s = s.apply(lambda x: 'other' if x not in s.value_counts()[:10].index else x)
        col = col + ' (Top 10 + other)'

    plt.subplot(grid_size, 2, 2*i+1)
    plt.title(col)
    sns.countplot(x=s,
                  order=s.value_counts().index,
                  hue=s,
                  hue_order=s.value_counts().index,
                  legend=False,
                  palette=palettes[i])
    plt.xlabel('')
    plt.xticks(rotation=90)


    plt.subplot(grid_size, 2, 2*i+2)
    plt.pie(s.value_counts(),
            labels=s.value_counts().index,
            autopct="%1.1f%%",
            explode=[0.01]*s.nunique(),
            colors=sns.color_palette(palettes[i],
                                     n_colors=len(s.value_counts().index)),
            radius=0.9)

plt.tight_layout()
plt.show()
```

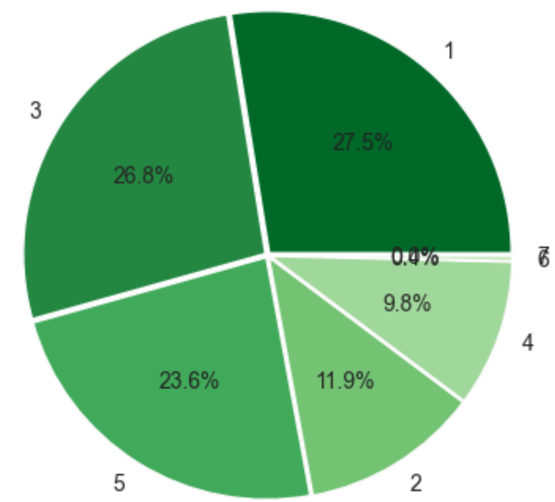

market_id
count
50000
40000
30000
20000
10000
0
2
4
1
3
5
6


2
30.4%
4
26.3%
0.4%
6
9.8%
5
21.1%
12.0%
1
3


store_primary_category (Top 10 + other)
count
70000
60000
50000
40000
30000
20000
10000
0


other
39.4%
4
10.3%
3.7%
28
9.0%
3.8%
55
4.6%
38
8.9%
4.7%
5.6%
5.1%
4.9%
24
46
39
13
58
20

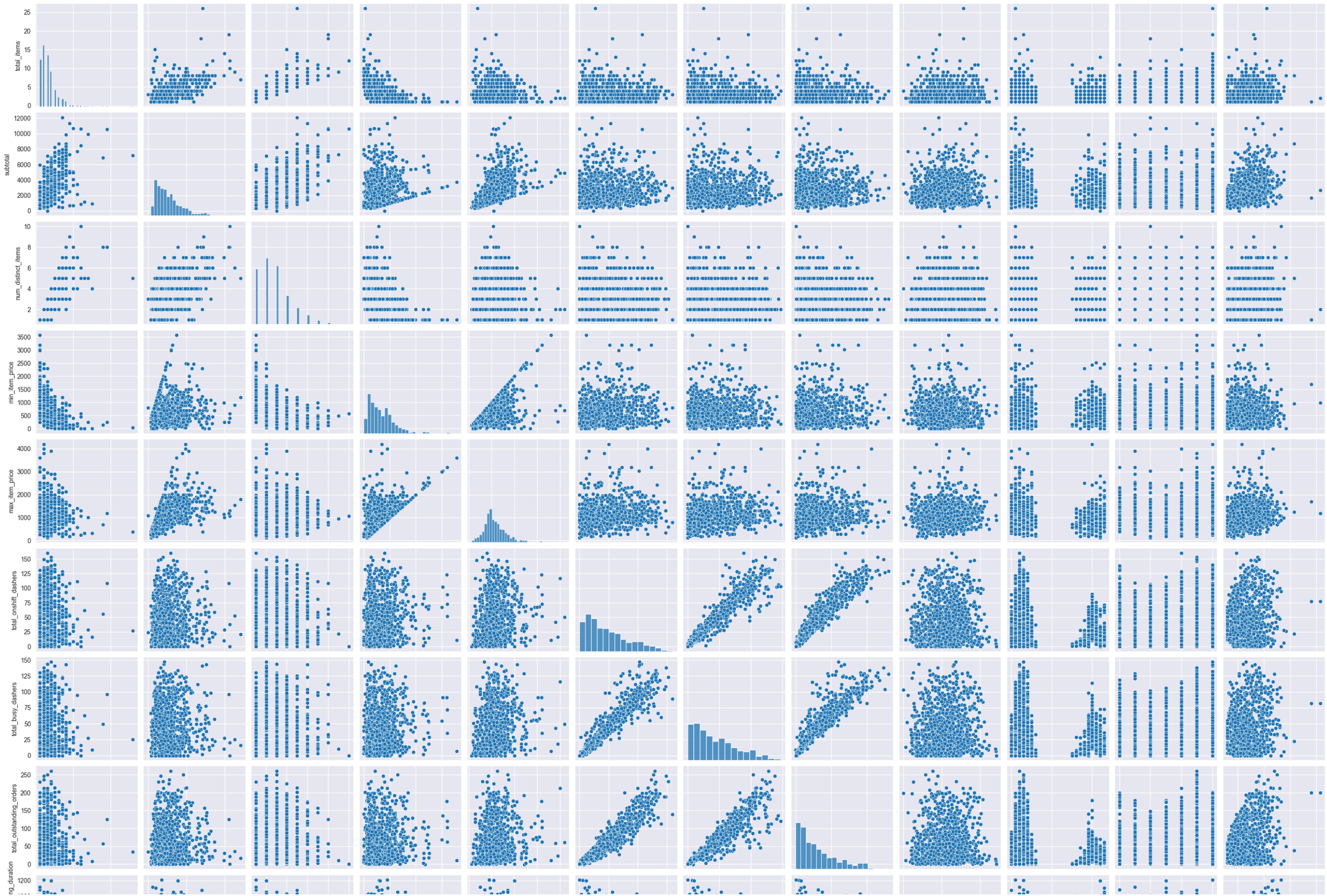




EDA - Multivariate Analysis

In [143…

```
df_sample = df.sample(1000)
sns.pairplot(df_sample)
plt.show()
```

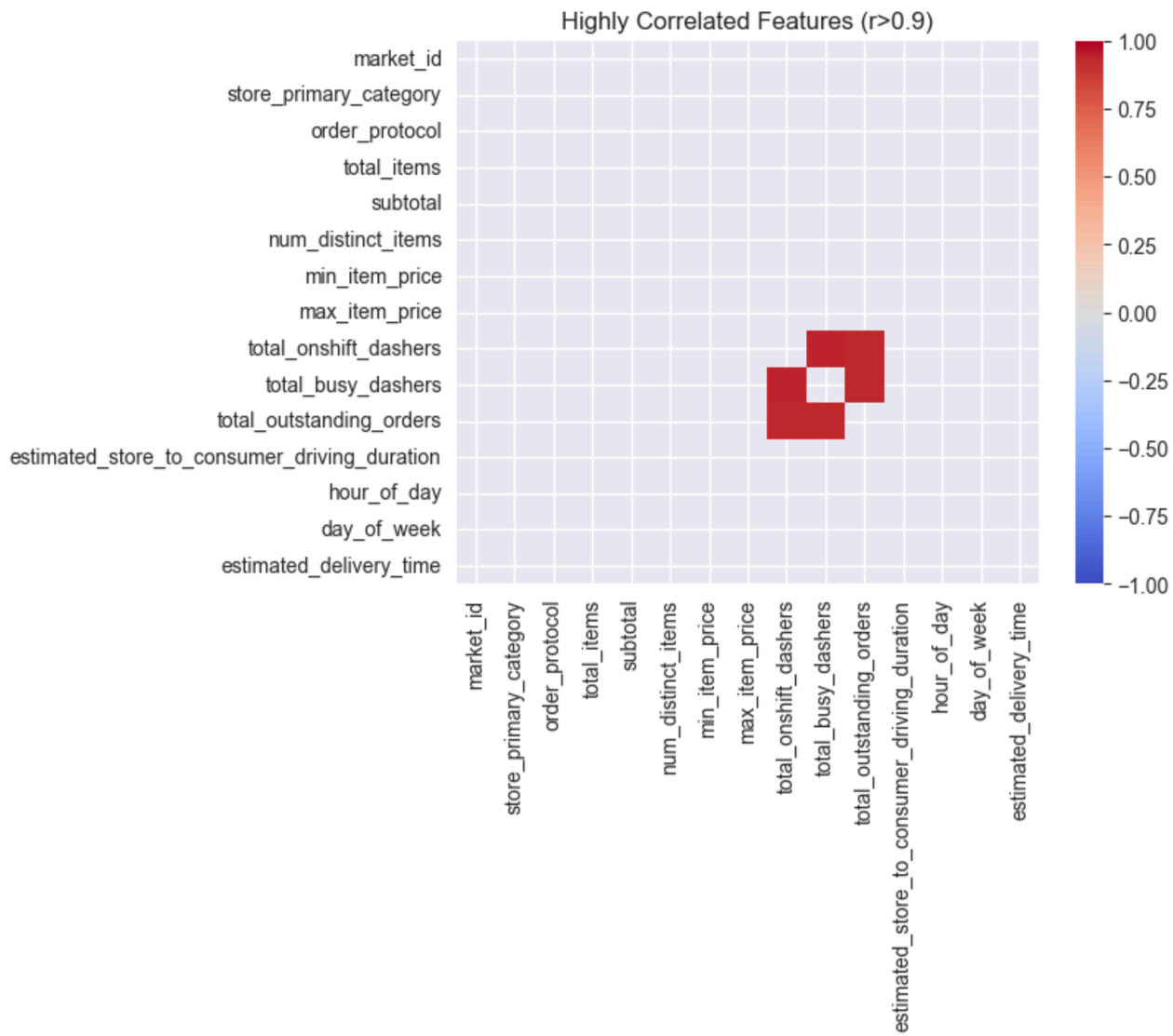

total_items
subtotal
num_distinct_items
min_item_price
max_item_price
total_onshift_dashers
total_busy_dashers
total_outstanding_orders

We can see that most of the features don't exhibit a clear linear pattern with each other.

Let us check for correlation.

In [146…

```
sns.heatmap(df.corr(),
            vmin=-1,
            vmax=1,
            cmap='coolwarm',
            mask=((abs(df.corr())<0.9)|(df.corr()==1))
            )
plt.title('Highly Correlated Features (r>0.9)')
plt.show()
```



The features `total_onshift_dashers`, `total_busy_dashers` and `total_outstanding_orders` are highly correlated with each other, with correlation coefficient > 0.9.

In [ ]:
```
# hour_of_day and day_of_week encoding
# cyclical variables - use sine cosine encoding

df["hour_of_day_sin"] = np.sin(2*np.pi*df["hour_of_day"]/24)
df["hour_of_day_cos"] = np.cos(2*np.pi*df["hour_of_day"]/24)

df['day_of_week_sin'] = np.sin(2*np.pi*df["day_of_week"]/7)
df['day_of_week_cos'] = np.cos(2*np.pi*df["day_of_week"]/7)
```

```
# We can now remove the original columns
df.drop(columns = ['hour_of_day','day_of_week'], inplace=True)
```

In [76]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 17 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  category
 1   store_primary_category                        175777 non-null  category
 2   order_protocol                                175777 non-null  category
 3   total_items                                   175777 non-null  int64
 4   subtotal                                      175777 non-null  float64
 5   num_distinct_items                            175777 non-null  int64
 6   min_item_price                                175777 non-null  float64
 7   max_item_price                                175777 non-null  float64
 8   total_onshift_dashers                         175777 non-null  float64
 9   total_busy_dashers                            175777 non-null  float64
 10  total_outstanding_orders                      175777 non-null  float64
 11  estimated_store_to_consumer_driving_duration  175777 non-null  float64
 12  estimated_delivery_time                       175777 non-null  float64
 13  hour_of_day_sin                               175777 non-null  float64
 14  hour_of_day_cos                               175777 non-null  float64
 15  day_of_week_sin                               175777 non-null  float64
 16  day_of_week_cos                               175777 non-null  float64
dtypes: category(3), float64(12), int64(2)
memory usage: 19.3 MB
```

Data Preparation for Modelling

In [77]:
```
# Split X and y
X = df.drop(columns=['estimated_delivery_time'])
y = df['estimated_delivery_time']
```

In [107…
```
# train, validation and test split

# first split: train vs temp (val+test)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# second split: validation vs test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
```

In [79]:
```
num_cols = X_train.select_dtypes(include='number').columns
cat_cols = X_train.select_dtypes(exclude='number').columns
print(f'NUMERICAL COLUMNS\n{num_cols}\n')
print(F'CATEGORICAL COLUMNS:\n{cat_cols}')
```

```
NUMERICAL COLUMNS
Index(['total_items', 'subtotal', 'num_distinct_items', 'min_item_price',
       'max_item_price', 'total_onshift_dashers', 'total_busy_dashers',
       'total_outstanding_orders',
       'estimated_store_to_consumer_driving_duration', 'hour_of_day_sin',
       'hour_of_day_cos', 'day_of_week_sin', 'day_of_week_cos'],
      dtype='object')

CATEGORICAL COLUMNS:
Index(['market_id', 'store_primary_category', 'order_protocol'], dtype='object')
```

Why scaling is required for Neural Networks?

Scaling is required because neural networks are sensitive to the magnitude of input features. If features have very different ranges, gradients can become unstable: some weights update too much, others too little. Scaling (like standardization or normalization) ensures:

- Faster convergence during training.
- Balanced weight updates across all features.
- Helps activation functions (like sigmoid, tanh, ReLU) operate in their effective range.
- Reduces risk of vanishing/exploding gradients.
- Without scaling, the network can take longer to train or fail to learn properly.

In [ ]:
```python
# Numerical feature scaling
scaler = StandardScaler()

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_val[num_cols] = scaler.transform(X_val[num_cols])
```

In [82]:
```python
# Convert categorical columns to integer indices
cat_maps = {}
cat_sizes = {}

for col in cat_cols:

    categories = df[col].unique()
    mapping = {k:v for v,k in enumerate(categories)}

    cat_maps[col] = mapping
    cat_sizes[col] = int(df[col].nunique())

    X_train[col] = X_train[col].map(mapping).cat.codes.astype('int32')
    X_val[col] = X_val[col].map(mapping).cat.codes.astype('int32')
    # X_test[col] = X_test[col].map(mapping).cat.codes.astype('int32')

print(f'===CATEGORICAL MAPPINGS===')
for k in cat_maps:
    print(f'{k}: {cat_maps[k]}')
print(f'\n===SIZES===')
for k in cat_sizes:
    print(f'{k}: {cat_sizes[k]}')
```

```
===CATEGORICAL MAPPINGS===
market_id: {1: 0, 2: 1, 3: 2, 4: 3, 5: 4, 6: 5}
store_primary_category: {4: 0, 46: 1, 36: 2, 38: 3, 58: 4, 68: 5, 15: 6, 57: 7, 55: 8, 20: 9, 60: 10, 13: 11, 45: 12, 39: 13, 34: 14, 18: 15, 29: 16, 23: 17, 50: 18, 70: 19, 6: 20, 7: 21, 10: 22, 2
8: 23, 24: 24, 61: 25, 59: 26, 72: 27, 16: 28, 65: 29, 47: 30, 53: 31, 49: 32, 40: 33, 66: 34, 42: 35, 21: 36, 14: 37, 35: 38, 11: 39, 52: 40, 2: 41, 71: 42, 25: 43, 54: 44, 69: 45, 26: 46, 12: 47,
32: 48, 30: 49, 17: 50, 33: 51, 22: 52, 31: 53, 0: 54, 51: 55, 48: 56, 67: 57, 44: 58, 62: 59, 9: 60, 27: 61, 19: 62, 1: 63, 5: 64, 41: 65, 37: 66, 64: 67, 56: 68, 63: 69, 43: 70, 8: 71, 3: 72}
order_protocol: {1: 0, 2: 1, 3: 2, 5: 3, 4: 4, 6: 5, 7: 6}

===SIZES===
market_id: 6
store_primary_category: 73
order_protocol: 7
```

Model Building

The following function defines the neural network architecture used for the regression task. The model is implemented using the **TensorFlow/Keras Functional API** to support multiple input types.

Input Structure

The network accepts two types of inputs:

1. Numerical Features
   - All numerical variables are standardized using `StandardScaler`.
   - These features are passed directly into the network as a dense input layer.
2. Categorical Features
   - Each categorical variable is processed using a learned embedding layer.
   - Embeddings allow the model to learn dense vector representations for categorical levels instead of relying on one-hot encoding.
   - The embedding dimension follows a common heuristic: `min(50, category_size // 2 + 1)`.

All numerical inputs and categorical embeddings are concatenated into a single feature vector before being passed into the dense network.

Hidden Layers

The dense portion of the network is dynamically defined and tuned using Keras Tuner. The following hyperparameters are explored:

- *Number of hidden layers:* 1 to 3
- *Units per layer:* 64 to 256 (step size 64)
- *Batch Normalization:* optionally applied after each dense layer
- *Dropout:* optional dropout regularization
- *Dropout rate:* 0.1 to 0.3

Each dense layer uses:

- *He Normal initialization* for better convergence with ReLU activations
- *ReLU activation*
- *Bias disabled when Batch Normalization is used*

Output Layer

The final layer is a single neuron with *linear activation*, suitable for a regression problem.

Optimization

The model is trained using:

- *Optimizer:* Adam
- *Loss function:* Mean Squared Error (MSE)
- *Evaluation metric:* Mean Absolute Error (MAE)

The learning rate of the optimizer is also treated as a hyperparameter and sampled on a **log scale** during tuning.

This flexible architecture allows the model to automatically search for the best network configuration for the dataset while efficiently handling both numerical and categorical features.

In [85]:
```
def build_model(hp):
    inputs = []
    embeddings = []

    # numerical input
    num_input = keras.layers.Input(shape=(len(num_cols),), name="num_input")
    inputs.append(num_input)
    embeddings.append(num_input)

    # Categorical Embeddings
    for col in cat_cols:
        inp = keras.Input(shape=(1,), name=f'{col}_input')
        emb_dim = min(50, cat_sizes[col]//2+1) # common heuristic
        emb = keras.layers.Embedding(input_dim=cat_sizes[col], output_dim=emb_dim)(inp)
        emb = keras.layers.Flatten()(emb)
        inputs.append(inp)
        embeddings.append(emb)

    x = keras.layers.Concatenate()(embeddings)

    # Dense layers
    for i in range(hp.Int("num_layers", 2, 3)):
        x = keras.layers.Dense(
            units=hp.Int(f"units_{i+1}", 64, 256, step=64),
            use_bias=False,
            kernel_initializer='he_normal'
        )(x)
        if hp.Boolean(f"batchnorm_{i+1}"):
            x = keras.layers.BatchNormalization()(x)
        x = keras.layers.Activation("relu")(x)
        if hp.Boolean(f"dropout_{i+1}"):
            x = keras.layers.Dropout(hp.Choice('dropout', [0.15,0.3]))(x)

    # Output
    output = keras.layers.Dense(1, activation="linear")(x)

    # Full model
    model = keras.Model(inputs=inputs, outputs=output)
```

```
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae"]
    )

    return model
```

Adam Optimizer

- Adam combines momentum and adaptive learning rates.
- Works well for sparse gradients and non-stationary objectives.
- Handles learning rate tuning automatically, reducing manual effort.
- Commonly chosen for regression because it converges faster and is robust to noisy gradients.

ReLU Activation Function

- ReLU (Rectified Linear Unit): `f(x) = max(0, x)`
- Advantages:
  - Avoids vanishing gradients (common in sigmoid/tanh).
  - Computationally efficient (simple thresholding).
  - Introduces non-linearity, enabling NN to learn complex relationships.
- For regression, it helps model positive and sparse activations efficiently.

He-Normal Kernel Initialization

- He initialization: weights sampled from `N(0, sqrt(2/fan_in))`.
- Works well with ReLU activations.
- Prevents vanishing/exploding gradients at the start of training.
- Ensures signal flows properly through deep layers.

Why Scaling is Required for Neural Networks

- Neural networks are sensitive to feature magnitude differences.
- Scaling (e.g., StandardScaler or MinMaxScaler):
  - Speeds up convergence.
  - Prevents some features dominating gradients.
  - Helps regularization methods like weight decay work properly.
- Essential when inputs vary widely (e.g., age in years vs. salary in lakhs).

Why Neural Network Performs Well on Large Datasets

- Neural networks have high capacity, i.e., can approximate complex functions.
- Large datasets:
  - Provide diverse examples for learning patterns.
  - Reduce overfitting risk by exposing the model to more variation.
  - Allow deep architectures to generalize better.

- Small datasets often lead to overfitting because NN memorizes instead of generalizing.

Let us now define inputs for model.

In [ ]:
```
train_inputs = {
    "num_input": X_train[num_cols].values
}

val_inputs = {
    "num_input": X_val[num_cols].values
}

for col in cat_cols:
    train_inputs[f"{col}_input"] = X_train[col].values
    val_inputs[f"{col}_input"] = X_val[col].values
```

Training Callbacks

Two callbacks are used during training to improve convergence and prevent unnecessary training.

Early Stopping

`EarlyStopping` monitors the validation loss during training. If the validation loss does not improve for a specified number of epochs (`patience`), training stops early. The option `restore_best_weights=True` ensures that the model weights from the best validation performance are retained. This helps prevent overfitting and reduces wasted computation.

Reduce Learning Rate on Plateau

`ReduceLROnPlateau` monitors the validation loss and reduces the learning rate when improvement stalls. If the validation loss does not improve for a certain number of epochs, the learning rate is multiplied by a factor (e.g., 0.3). Lowering the learning rate allows the optimizer to make finer updates and often helps the model converge to a better minimum.

In [87]:
```
callbacks = [

    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        restore_best_weights=True,
        verbose=1
    ),

    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.3,
        patience=5,
        min_lr=1e-6,
        verbose=1
    )
]
```

We use keras' `Hyperband` as Hyperparameter Tuning tool.

In [88]:
```
tuner = kt.Hyperband(
    build_model,
```

```
    objective="val_loss",
    max_epochs=100,
    factor=3,
    directory="tuner_dir",
    project_name="tabular_ann"
)
```

Let us now search for Hyperparameters.

In [89]:
```
tuner.search(
    train_inputs,
    y_train,
    validation_data=(val_inputs, y_val),
    epochs=100,
    batch_size=256,
    callbacks = callbacks
)
```

```
Trial 145 Complete [00h 00m 49s]
val_loss: 0.15138332545757294

Best val_loss So Far: 0.14648333191871643
Total elapsed time: 00h 20m 29s

Search: Running Trial #146

Value             |Best Value So Far |Hyperparameter
3                 |2                 |num_layers
256               |256               |units_1
True              |False             |batchnorm_1
True              |False             |dropout_1
256               |256               |units_2
True              |False             |batchnorm_2
False             |False             |dropout_2
0.15              |0.3               |dropout
256               |128               |units_3
True              |True              |batchnorm_3
False             |True              |dropout_3
34                |34                |tuner/epochs
12                |12                |tuner/initial_epoch
4                 |4                 |tuner/bracket
3                 |3                 |tuner/round
0137              |0140              |tuner/trial_id

Epoch 13/34
```

```
c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\saving\saving_lib.py:797: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables wh
ereas the saved optimizer has 30 variables.
  saveable.load_own_variables(weights_store.get(inner_path))
```

```
481/481 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 1.3721 - mae: 0.9039 - val_loss: 0.4768 - val_mae: 0.5444 - learning_rate: 0.0010
Epoch 14/34
481/481 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 1.2246 - mae: 0.8530 - val_loss: 0.5287 - val_mae: 0.5860 - learning_rate: 0.0010
Epoch 15/34
481/481 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 1.1912 - mae: 0.8471 - val_loss: 0.3514 - val_mae: 0.4677 - learning_rate: 0.0010
Epoch 16/34
481/481 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 1.1607 - mae: 0.8350 - val_loss: 0.5076 - val_mae: 0.5749 - learning_rate: 0.0010
Epoch 17/34
481/481 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 1.1416 - mae: 0.8286 - val_loss: 0.5821 - val_mae: 0.6070 - learning_rate: 0.0010
Epoch 18/34
481/481 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 1.0851 - mae: 0.8103 - val_loss: 0.5356 - val_mae: 0.5770 - learning_rate: 0.0010
Epoch 19/34
481/481 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 1.0135 - mae: 0.7797 - val_loss: 0.2309 - val_mae: 0.3796 - learning_rate: 0.0010
Epoch 20/34
302/481 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.9629 - mae: 0.7574
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[89], line 1
----> 1 tuner.search(
      2     train_inputs,
      3     y_train,
      4     validation_data=(val_inputs, y_val),
      5     epochs=100,
      6     batch_size=256,
      7     callbacks = callbacks
      8 )

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\engine\base_tuner.py:234, in BaseTuner.search(self, *fit_args, **fit_kwargs)
    231         continue
    233     self.on_trial_begin(trial)
--> 234     self._try_run_and_update_trial(trial, *fit_args, **fit_kwargs)
    235     self.on_trial_end(trial)
    236 self.on_search_end()

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\engine\base_tuner.py:274, in BaseTuner._try_run_and_update_trial(self, trial, *fit_args, **fit_kwargs)
    272 def _try_run_and_update_trial(self, trial, *fit_args, **fit_kwargs):
    273     try:
--> 274         self._run_and_update_trial(trial, *fit_args, **fit_kwargs)
    275         trial.status = trial_module.TrialStatus.COMPLETED
    276         return

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\engine\base_tuner.py:239, in BaseTuner._run_and_update_trial(self, trial, *fit_args, **fit_kwargs)
    238 def _run_and_update_trial(self, trial, *fit_args, **fit_kwargs):
--> 239     results = self.run_trial(trial, *fit_args, **fit_kwargs)
    240     if self.oracle.get_trial(trial.trial_id).metrics.exists(
    241         self.oracle.objective.name
    242     ):
    243         # The oracle is updated by calling `self.oracle.update_trial()` in
    244         # `Tuner.run_trial()`. For backward compatibility, we support this
    245         # use case. No further action needed in this case.
    246         warnings.warn(
    247             "The use case of calling "
    248             "`self.oracle.update_trial(trial_id, metrics)` "
   (...)    254             stacklevel=2,
    255         )

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\tuners\hyperband.py:427, in Hyperband.run_trial(self, trial, *fit_args, **fit_kwargs)
    425     fit_kwargs["epochs"] = hp.values["tuner/epochs"]
    426     fit_kwargs["initial_epoch"] = hp.values["tuner/initial_epoch"]
--> 427 return super().run_trial(trial, *fit_args, **fit_kwargs)

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\engine\tuner.py:314, in Tuner.run_trial(self, trial, *args, **kwargs)
    312     callbacks.append(model_checkpoint)
    313     copied_kwargs["callbacks"] = callbacks
--> 314     obj_value = self._build_and_fit_model(trial, *args, **copied_kwargs)
    316     histories.append(obj_value)
    317 return histories

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\engine\tuner.py:233, in Tuner._build_and_fit_model(self, trial, *args, **kwargs)
    231 hp = trial.hyperparameters
    232 model = self._try_build(hp)
```

```
--> 233 results = self.hypermodel.fit(hp, model, *args, **kwargs)
    235 # Save the build config for model loading later.
    236 if backend.config.multi_backend():

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras_tuner\src\engine\hypermodel.py:149, in HyperModel.fit(self, hp, model, *args, **kwargs)
    125 def fit(self, hp, model, *args, **kwargs):
    126     """Train the model.
    127
    128     Args:
   (...)    147         If return a float, it should be the `objective` value.
    148     """
--> 149     return model.fit(*args, **kwargs)

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\utils\traceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\backend\tensorflow\trainer.py:399, in TensorFlowTrainer.fit(self, x, y, batch_size, epochs, verbose, callbacks,
validation_split, validation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch, validation_steps, validation_batch_size, validation_freq)
    397 for begin_step, end_step, iterator in epoch_iterator:
    398     callbacks.on_train_batch_begin(begin_step)
--> 399     logs = self.train_function(iterator)
    400     callbacks.on_train_batch_end(end_step, logs)
    401     if self.stop_training:

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\backend\tensorflow\trainer.py:241, in TensorFlowTrainer._make_function.<locals>.function(iterator)
    237 def function(iterator):
    238     if isinstance(
    239         iterator, (tf.data.Iterator, tf.distribute.DistributedIterator)
    240     ):
--> 241         opt_outputs = multi_step_on_iterator(iterator)
    242         if not opt_outputs.has_value():
    243             raise StopIteration

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\util\traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    148 filtered_tb = None
    149 try:
--> 150   return fn(*args, **kwargs)
    151 except Exception as e:
    152   filtered_tb = _process_traceback_frames(e.__traceback__)

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\polymorphic_function\polymorphic_function.py:833, in Function.__call__(self, *args, **kwds)
    830 compiler = "xla" if self._jit_compile else "nonXla"
    832 with OptionalXlaContext(self._jit_compile):
--> 833   result = self._call(*args, **kwds)
    835 new_tracing_count = self.experimental_get_tracing_count()
    836 without_tracing = (tracing_count == new_tracing_count)

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\polymorphic_function\polymorphic_function.py:878, in Function._call(self, *args, **kwds)
    875 self._lock.release()
    876 # In this case we have not created variables on the first call. So we can
    877 # run the first trace but we should fail if variables are created.
--> 878 results = tracing_compilation.call_function(
```

```
    879     args, kwds, self._variable_creation_config
    880 )
    881 if self._created_variables:
    882   raise ValueError("Creating variables on a non-first call to a function"
    883                    " decorated with tf.function.")

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\polymorphic_function\tracing_compilation.py:139, in call_function(args, kwargs, tracing_options)
    137 bound_args = function.function_type.bind(*args, **kwargs)
    138 flat_inputs = function.function_type.unpack_inputs(bound_args)
--> 139 return function._call_flat(  # pylint: disable=protected-access
    140     flat_inputs, captured_inputs=function.captured_inputs
    141 )

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\polymorphic_function\concrete_function.py:1322, in ConcreteFunction._call_flat(self, tensor_input
s, captured_inputs)
   1318 possible_gradient_type = gradients_util.PossibleTapeGradientTypes(args)
   1319 if (possible_gradient_type == gradients_util.POSSIBLE_GRADIENT_TYPES_NONE
   1320     and executing_eagerly):
   1321   # No tape is watching; skip to running the function.
-> 1322   return self._inference_function.call_preflattened(args)
   1323 forward_backward = self._select_forward_and_backward_functions(
   1324     args,
   1325     possible_gradient_type,
   1326     executing_eagerly)
   1327 forward_function, args_with_tangents = forward_backward.forward()

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\polymorphic_function\atomic_function.py:216, in AtomicFunction.call_preflattened(self, args)
    214 def call_preflattened(self, args: Sequence[core.Tensor]) -> Any:
    215   """Calls with flattened tensor inputs and returns the structured output."""
--> 216   flat_outputs = self.call_flat(*args)
    217   return self.function_type.pack_output(flat_outputs)

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\polymorphic_function\atomic_function.py:251, in AtomicFunction.call_flat(self, *args)
    249 with record.stop_recording():
    250   if self._bound_context.executing_eagerly():
--> 251     outputs = self._bound_context.call_function(
    252         self.name,
    253         list(args),
    254         len(self.function_type.flat_outputs),
    255     )
    256   else:
    257     outputs = make_call_op_in_graph(
    258         self,
    259         list(args),
    260         self._bound_context.function_call_options.as_attrs(),
    261     )

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\context.py:1688, in Context.call_function(self, name, tensor_inputs, num_outputs)
   1686 cancellation_context = cancellation.context()
   1687 if cancellation_context is None:
-> 1688   outputs = execute.execute(
   1689       name.decode(       ),
   1690       num_outputs=num_outputs,
   1691       inputs=tensor_inputs,
   1692       attrs=attrs,
   1693       ctx=self,
```

```
   1694   )
   1695 else:
   1696   outputs = execute.execute_with_cancellation(
   1697       name.decode("utf-8"),
   1698       num_outputs=num_outputs,
   (...)   1702       cancellation_manager=cancellation_context,
   1703   )

File c:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\eager\execute.py:53, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)
     51 try:
     52   ctx.ensure_initialized()
---> 53   tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
     54                                       inputs, attrs, num_outputs)
     55 except core._NotOkStatusException as e:
     56   if name is not None:

KeyboardInterrupt:
```

In [90]:
```
best_hp = tuner.get_best_hyperparameters(1)[0]
```

These are the best hyperparameters.

In [91]:
```
best_hp.values
```

Out[91]:
```
{'num_layers': 2,
 'units_1': 256,
 'batchnorm_1': False,
 'dropout_1': False,
 'units_2': 256,
 'batchnorm_2': False,
 'dropout_2': False,
 'dropout': 0.3,
 'units_3': 128,
 'batchnorm_3': True,
 'dropout_3': True,
 'tuner/epochs': 34,
 'tuner/initial_epoch': 12,
 'tuner/bracket': 4,
 'tuner/round': 3,
 'tuner/trial_id': '0140'}
```

In [92]:
```
best_model = tuner.hypermodel.build(best_hp)
```

In [93]:
```
best_model.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
market_id_input (InputLayer)	(None, 1)	0	-
store_primary_cate… (InputLayer)	(None, 1)	0	-
order_protocol_inp… (InputLayer)	(None, 1)	0	-
embedding_3 (Embedding)	(None, 1, 4)	24	market_id_input[…
embedding_4 (Embedding)	(None, 1, 37)	2,701	store_primary_ca…
embedding_5 (Embedding)	(None, 1, 4)	28	order_protocol_i…
num_input (InputLayer)	(None, 13)	0	-
flatten_3 (Flatten)	(None, 4)	0	embedding_3[0][0]
flatten_4 (Flatten)	(None, 37)	0	embedding_4[0][0]
flatten_5 (Flatten)	(None, 4)	0	embedding_5[0][0]
concatenate_1 (Concatenate)	(None, 58)	0	num_input[0][0], flatten_3[0][0], flatten_4[0][0], flatten_5[0][0]
dense_4 (Dense)	(None, 256)	14,848	concatenate_1[0]…
activation_3 (Activation)	(None, 256)	0	dense_4[0][0]
dense_5 (Dense)	(None, 256)	65,536	activation_3[0][…
activation_4 (Activation)	(None, 256)	0	dense_5[0][0]
dense_6 (Dense)	(None, 1)	257	activation_4[0][…

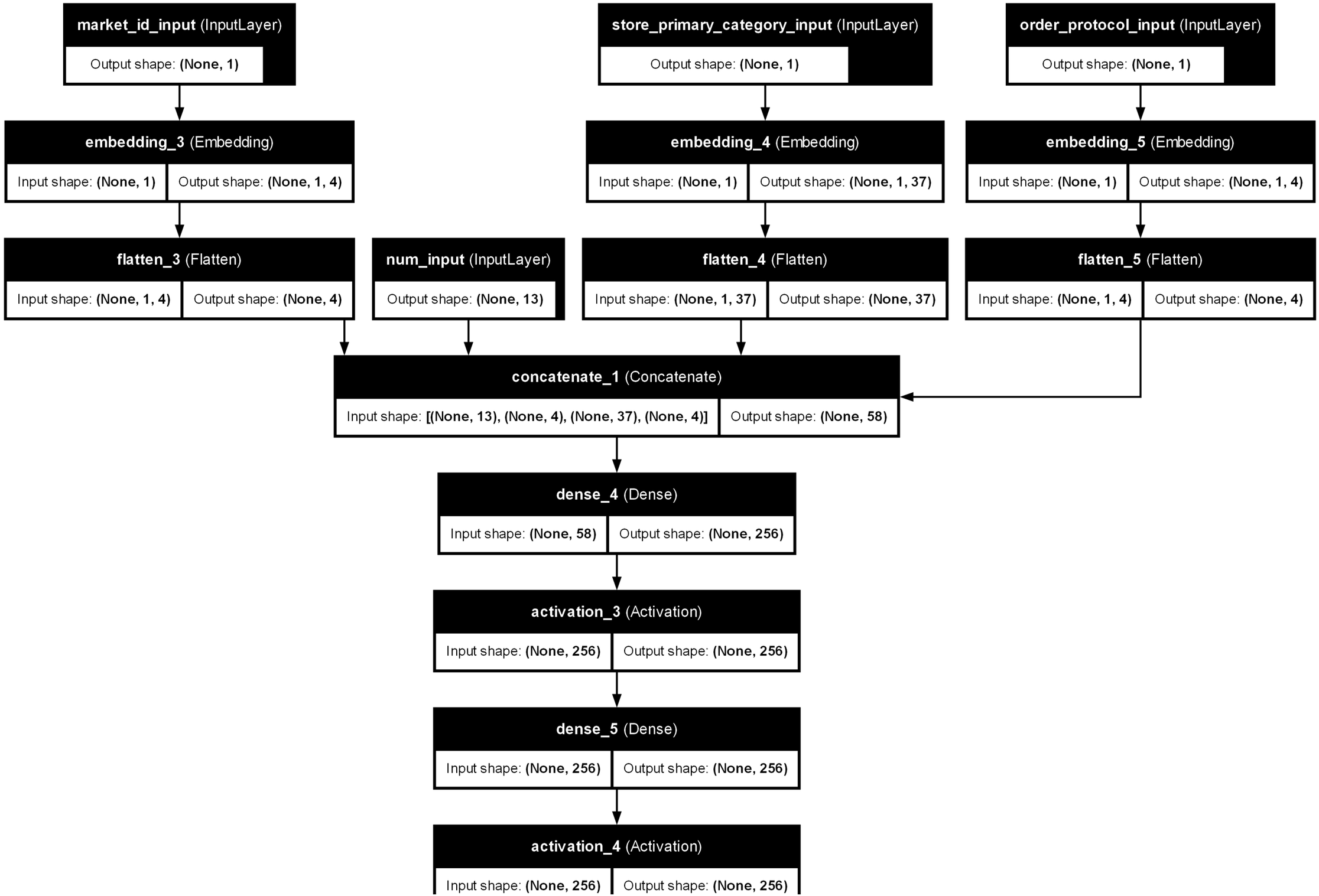
Total params: 83,394 (325.76 KB)

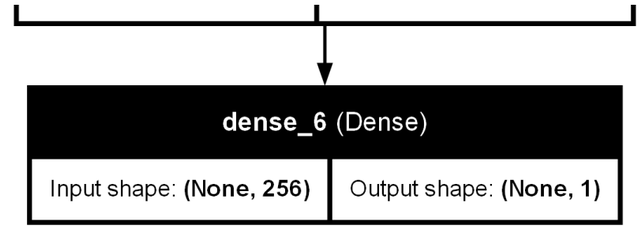
Trainable params: 83,394 (325.76 KB)

Non-trainable params: 0 (0.00 B)

In [94]:
```
plot_model(best_model, to_file='trained_model.png', show_shapes=True, show_layer_names=True)
```

Out[94]:


market_id_input (InputLayer)
Output shape: (None, 1)
store_primary_category_input (InputLayer)
Output shape: (None, 1)
order_protocol_input (InputLayer)
Output shape: (None, 1)
embedding_3 (Embedding)
Input shape: (None, 1)
Output shape: (None, 1, 4)
embedding_4 (Embedding)
Input shape: (None, 1)
Output shape: (None, 1, 37)
embedding_5 (Embedding)
Input shape: (None, 1)
Output shape: (None, 1, 4)
flatten_3 (Flatten)
Input shape: (None, 1, 4)
Output shape: (None, 4)
num_input (InputLayer)
Output shape: (None, 13)
flatten_4 (Flatten)
Input shape: (None, 1, 37)
Output shape: (None, 37)
flatten_5 (Flatten)
Input shape: (None, 1, 4)
Output shape: (None, 4)
concatenate_1 (Concatenate)
Input shape: [(None, 13), (None, 4), (None, 37), (None, 4)]
Output shape: (None, 58)
dense_4 (Dense)
Input shape: (None, 58)
Output shape: (None, 256)
activation_3 (Activation)
Input shape: (None, 256)
Output shape: (None, 256)
dense_5 (Dense)
Input shape: (None, 256)
Output shape: (None, 256)
activation_4 (Activation)
Input shape: (None, 256)
Output shape: (None, 256)



Let us now train the full model with best performing hyperparameters:

In [95]:
```
history = best_model.fit(train_inputs,
               y_train,
               validation_data=(val_inputs, y_val),
               epochs=100,
               callbacks=callbacks,
               verbose=1)
```

```
Epoch 1/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 19.4596 - mae: 2.0610 - val_loss: 2.2293 - val_mae: 1.1468 - learning_rate: 0.0010
Epoch 2/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 1.1516 - mae: 0.8088 - val_loss: 0.5891 - val_mae: 0.5820 - learning_rate: 0.0010
Epoch 3/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.5383 - mae: 0.5617 - val_loss: 0.5954 - val_mae: 0.6217 - learning_rate: 0.0010
Epoch 4/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.3781 - mae: 0.4776 - val_loss: 0.3301 - val_mae: 0.4529 - learning_rate: 0.0010
Epoch 5/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.3232 - mae: 0.4423 - val_loss: 0.3023 - val_mae: 0.4367 - learning_rate: 0.0010
Epoch 6/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2749 - mae: 0.4111 - val_loss: 0.2781 - val_mae: 0.4169 - learning_rate: 0.0010
Epoch 7/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2597 - mae: 0.3980 - val_loss: 0.2639 - val_mae: 0.4098 - learning_rate: 0.0010
Epoch 8/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2436 - mae: 0.3878 - val_loss: 0.2120 - val_mae: 0.3669 - learning_rate: 0.0010
Epoch 9/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2290 - mae: 0.3763 - val_loss: 0.2388 - val_mae: 0.3889 - learning_rate: 0.0010
Epoch 10/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2230 - mae: 0.3711 - val_loss: 0.2818 - val_mae: 0.4220 - learning_rate: 0.0010
Epoch 11/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2097 - mae: 0.3615 - val_loss: 0.2232 - val_mae: 0.3751 - learning_rate: 0.0010
Epoch 12/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.2110 - mae: 0.3606 - val_loss: 0.1805 - val_mae: 0.3405 - learning_rate: 0.0010
Epoch 13/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.1978 - mae: 0.3510 - val_loss: 0.3073 - val_mae: 0.4496 - learning_rate: 0.0010
Epoch 14/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.1966 - mae: 0.3500 - val_loss: 0.2916 - val_mae: 0.4336 - learning_rate: 0.0010
Epoch 15/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.1885 - mae: 0.3440 - val_loss: 0.1458 - val_mae: 0.3097 - learning_rate: 0.0010
Epoch 16/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1876 - mae: 0.3433 - val_loss: 0.2132 - val_mae: 0.3654 - learning_rate: 0.0010
Epoch 17/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.1802 - mae: 0.3370 - val_loss: 0.1733 - val_mae: 0.3349 - learning_rate: 0.0010
Epoch 18/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1822 - mae: 0.3376 - val_loss: 0.1664 - val_mae: 0.3288 - learning_rate: 0.0010
Epoch 19/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1781 - mae: 0.3346 - val_loss: 0.2789 - val_mae: 0.4161 - learning_rate: 0.0010
Epoch 20/100
3842/3846 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.1863 - mae: 0.3391
Epoch 20: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.1755 - mae: 0.3322 - val_loss: 0.1665 - val_mae: 0.3309 - learning_rate: 0.0010
Epoch 21/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1132 - mae: 0.2754 - val_loss: 0.1311 - val_mae: 0.2964 - learning_rate: 3.0000e-04
Epoch 22/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.1121 - mae: 0.2741 - val_loss: 0.1108 - val_mae: 0.2757 - learning_rate: 3.0000e-04
Epoch 23/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1114 - mae: 0.2736 - val_loss: 0.1954 - val_mae: 0.3552 - learning_rate: 3.0000e-04
Epoch 24/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 11s 2ms/step - loss: 0.1104 - mae: 0.2719 - val_loss: 0.1881 - val_mae: 0.3528 - learning_rate: 3.0000e-04
Epoch 25/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 10s 2ms/step - loss: 0.1100 - mae: 0.2717 - val_loss: 0.1369 - val_mae: 0.3019 - learning_rate: 3.0000e-04
Epoch 26/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1090 - mae: 0.2709 - val_loss: 0.1120 - val_mae: 0.2773 - learning_rate: 3.0000e-04
Epoch 27/100
3830/3846 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.1072 - mae: 0.2689
```

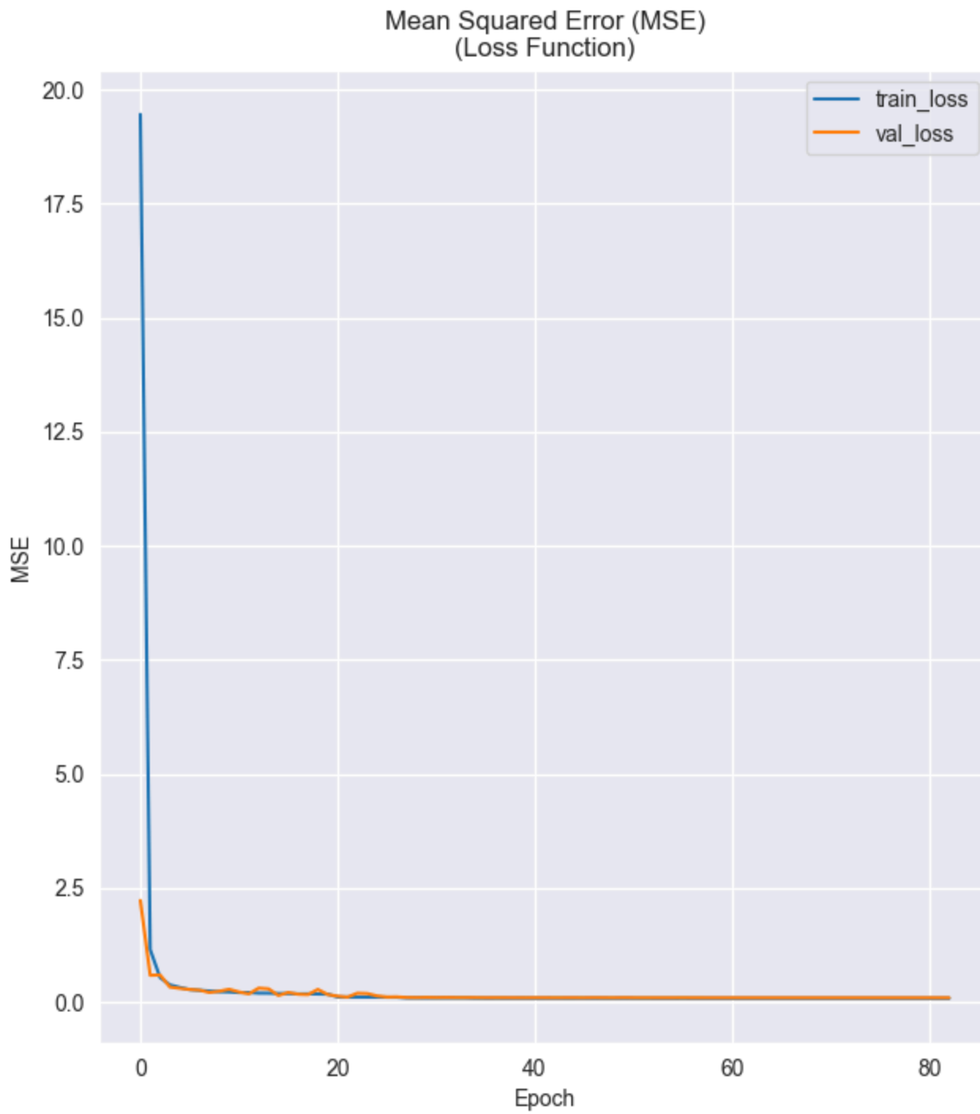
```
Epoch 27: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.1084 - mae: 0.2705 - val_loss: 0.1150 - val_mae: 0.2803 - learning_rate: 3.0000e-04
Epoch 28/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0931 - mae: 0.2538 - val_loss: 0.1028 - val_mae: 0.2681 - learning_rate: 9.0000e-05
Epoch 29/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0930 - mae: 0.2537 - val_loss: 0.1003 - val_mae: 0.2653 - learning_rate: 9.0000e-05
Epoch 30/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0922 - mae: 0.2528 - val_loss: 0.1037 - val_mae: 0.2686 - learning_rate: 9.0000e-05
Epoch 31/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0920 - mae: 0.2527 - val_loss: 0.1005 - val_mae: 0.2658 - learning_rate: 9.0000e-05
Epoch 32/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0921 - mae: 0.2526 - val_loss: 0.1024 - val_mae: 0.2674 - learning_rate: 9.0000e-05
Epoch 33/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0920 - mae: 0.2524 - val_loss: 0.1005 - val_mae: 0.2657 - learning_rate: 9.0000e-05
Epoch 34/100
3822/3846 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0909 - mae: 0.2517
Epoch 34: ReduceLROnPlateau reducing learning rate to 2.700000040931627e-05.
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0918 - mae: 0.2525 - val_loss: 0.1006 - val_mae: 0.2659 - learning_rate: 9.0000e-05
Epoch 35/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0870 - mae: 0.2469 - val_loss: 0.0995 - val_mae: 0.2645 - learning_rate: 2.7000e-05
Epoch 36/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0868 - mae: 0.2467 - val_loss: 0.0974 - val_mae: 0.2624 - learning_rate: 2.7000e-05
Epoch 37/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0868 - mae: 0.2466 - val_loss: 0.0979 - val_mae: 0.2631 - learning_rate: 2.7000e-05
Epoch 38/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0868 - mae: 0.2467 - val_loss: 0.0977 - val_mae: 0.2630 - learning_rate: 2.7000e-05
Epoch 39/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - loss: 0.0867 - mae: 0.2465 - val_loss: 0.0972 - val_mae: 0.2624 - learning_rate: 2.7000e-05
Epoch 40/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0866 - mae: 0.2464 - val_loss: 0.0970 - val_mae: 0.2622 - learning_rate: 2.7000e-05
Epoch 41/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0865 - mae: 0.2463 - val_loss: 0.0975 - val_mae: 0.2629 - learning_rate: 2.7000e-05
Epoch 42/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0865 - mae: 0.2463 - val_loss: 0.0973 - val_mae: 0.2626 - learning_rate: 2.7000e-05
Epoch 43/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0865 - mae: 0.2463 - val_loss: 0.0969 - val_mae: 0.2621 - learning_rate: 2.7000e-05
Epoch 44/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0863 - mae: 0.2461 - val_loss: 0.0971 - val_mae: 0.2624 - learning_rate: 2.7000e-05
Epoch 45/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0865 - mae: 0.2461 - val_loss: 0.0972 - val_mae: 0.2623 - learning_rate: 2.7000e-05
Epoch 46/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0863 - mae: 0.2461 - val_loss: 0.0967 - val_mae: 0.2618 - learning_rate: 2.7000e-05
Epoch 47/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0863 - mae: 0.2460 - val_loss: 0.0986 - val_mae: 0.2637 - learning_rate: 2.7000e-05
Epoch 48/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0862 - mae: 0.2458 - val_loss: 0.0973 - val_mae: 0.2626 - learning_rate: 2.7000e-05
Epoch 49/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0861 - mae: 0.2458 - val_loss: 0.0969 - val_mae: 0.2620 - learning_rate: 2.7000e-05
Epoch 50/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0861 - mae: 0.2458 - val_loss: 0.0987 - val_mae: 0.2639 - learning_rate: 2.7000e-05
Epoch 51/100
3845/3846 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0863 - mae: 0.2456
Epoch 51: ReduceLROnPlateau reducing learning rate to 8.100000013655517e-06.
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0862 - mae: 0.2458 - val_loss: 0.0971 - val_mae: 0.2621 - learning_rate: 2.7000e-05
Epoch 52/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0846 - mae: 0.2441 - val_loss: 0.0963 - val_mae: 0.2614 - learning_rate: 8.1000e-06
```

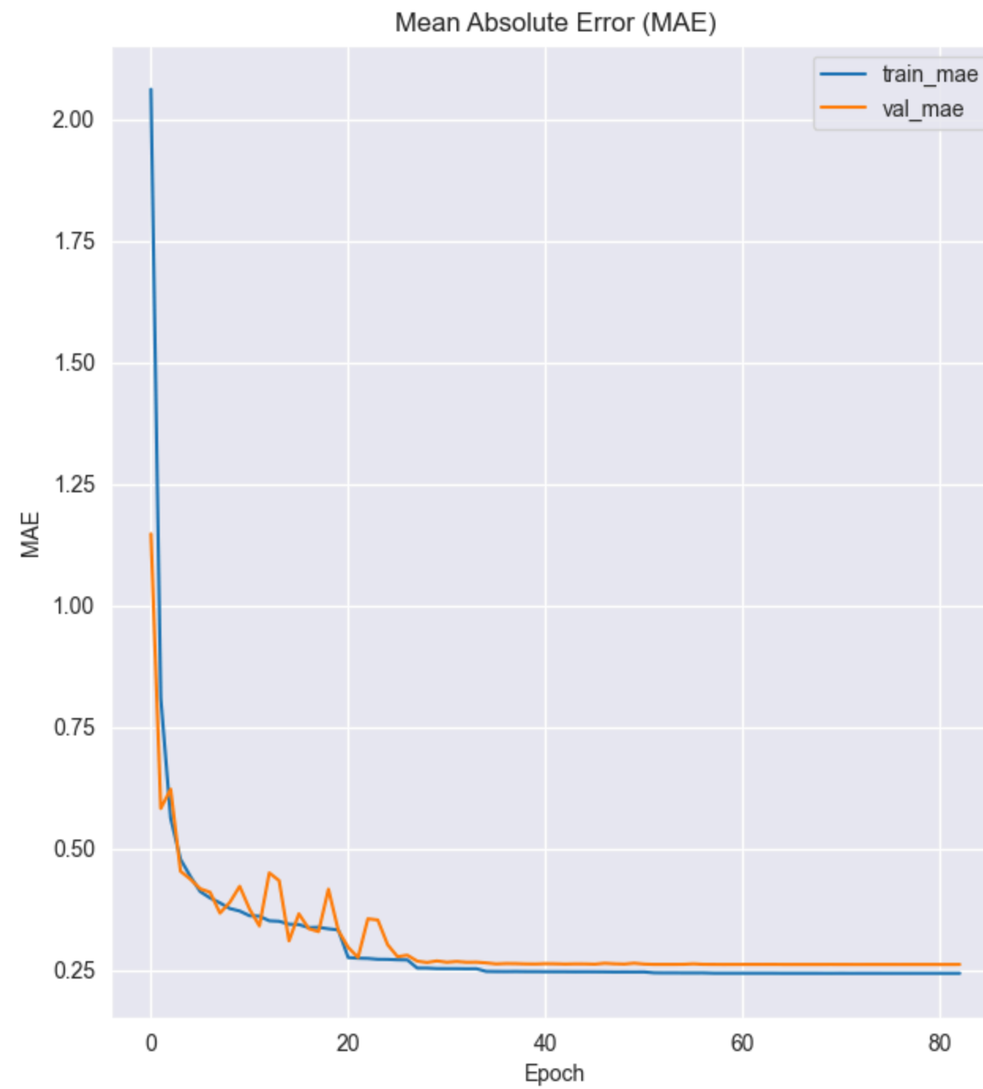
```
Epoch 53/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0845 - mae: 0.2439 - val_loss: 0.0964 - val_mae: 0.2614 - learning_rate: 8.1000e-06
Epoch 54/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0846 - mae: 0.2439 - val_loss: 0.0964 - val_mae: 0.2614 - learning_rate: 8.1000e-06
Epoch 55/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0845 - mae: 0.2438 - val_loss: 0.0965 - val_mae: 0.2616 - learning_rate: 8.1000e-06
Epoch 56/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0845 - mae: 0.2438 - val_loss: 0.0975 - val_mae: 0.2625 - learning_rate: 8.1000e-06
Epoch 57/100
3839/3846 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0851 - mae: 0.2447
Epoch 57: ReduceLROnPlateau reducing learning rate to 2.429999949526973e-06.
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0845 - mae: 0.2439 - val_loss: 0.0964 - val_mae: 0.2615 - learning_rate: 8.1000e-06
Epoch 58/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2433 - val_loss: 0.0962 - val_mae: 0.2614 - learning_rate: 2.4300e-06
Epoch 59/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2433 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 2.4300e-06
Epoch 60/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2433 - val_loss: 0.0963 - val_mae: 0.2614 - learning_rate: 2.4300e-06
Epoch 61/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2433 - val_loss: 0.0962 - val_mae: 0.2614 - learning_rate: 2.4300e-06
Epoch 62/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2433 - val_loss: 0.0963 - val_mae: 0.2614 - learning_rate: 2.4300e-06
Epoch 63/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2433 - val_loss: 0.0962 - val_mae: 0.2614 - learning_rate: 2.4300e-06
Epoch 64/100
3845/3846 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0843 - mae: 0.2429
Epoch 64: ReduceLROnPlateau reducing learning rate to 1e-06.
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0840 - mae: 0.2432 - val_loss: 0.0963 - val_mae: 0.2614 - learning_rate: 2.4300e-06
Epoch 65/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0839 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 66/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0839 - mae: 0.2431 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 67/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0839 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 68/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 16s 4ms/step - loss: 0.0839 - mae: 0.2431 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 69/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 70/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 17s 4ms/step - loss: 0.0838 - mae: 0.2430 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 71/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 13s 3ms/step - loss: 0.0839 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 72/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 73/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 74/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0839 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 75/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 76/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 77/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 78/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
```

```
Epoch 79/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0961 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 80/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - loss: 0.0838 - mae: 0.2431 - val_loss: 0.0962 - val_mae: 0.2613 - learning_rate: 1.0000e-06
Epoch 81/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0838 - mae: 0.2430 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 82/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0838 - mae: 0.2430 - val_loss: 0.0961 - val_mae: 0.2612 - learning_rate: 1.0000e-06
Epoch 83/100
3846/3846 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - loss: 0.0838 - mae: 0.2430 - val_loss: 0.0962 - val_mae: 0.2614 - learning_rate: 1.0000e-06
Epoch 83: early stopping
Restoring model weights from the end of the best epoch: 73.
```

In [96]:

```
plt.figure(figsize=(16,8))

plt.subplot(1,2,1)
plt.title('Mean Squared Error (MSE)\n(Loss Function)')
plt.plot(history.history["loss"], label="train_loss")
plt.plot(history.history["val_loss"], label="val_loss")
plt.legend()
plt.xlabel("Epoch")
plt.ylabel("MSE")

plt.subplot(1,2,2)
plt.title('Mean Absolute Error (MAE)')
plt.plot(history.history['mae'], label="train_mae")
plt.plot(history.history["val_mae"], label="val_mae")
plt.legend()
plt.xlabel("Epoch")
plt.ylabel("MAE")

plt.show()
```





Training and Validation Performance

The training and validation curves for **Mean Squared Error (MSE)** and **Mean Absolute Error (MAE)** show a stable learning process.

Both metrics decrease rapidly during the initial epochs, indicating that the model quickly learns the major patterns in the data. After approximately 25–30 epochs, the curves begin to plateau, suggesting that the model has largely converged and additional training provides minimal improvement.

The training and validation curves remain very close throughout the training process, with no significant divergence. This indicates that the model generalizes well to unseen data and does not exhibit signs of overfitting. Minor fluctuations in the validation curve during early epochs are expected and are typically caused by stochastic minibatch training and dropout regularization.

Overall, the smooth convergence and small gap between training and validation metrics suggest that the chosen architecture, regularization strategy, and optimization settings are appropriate for the problem.

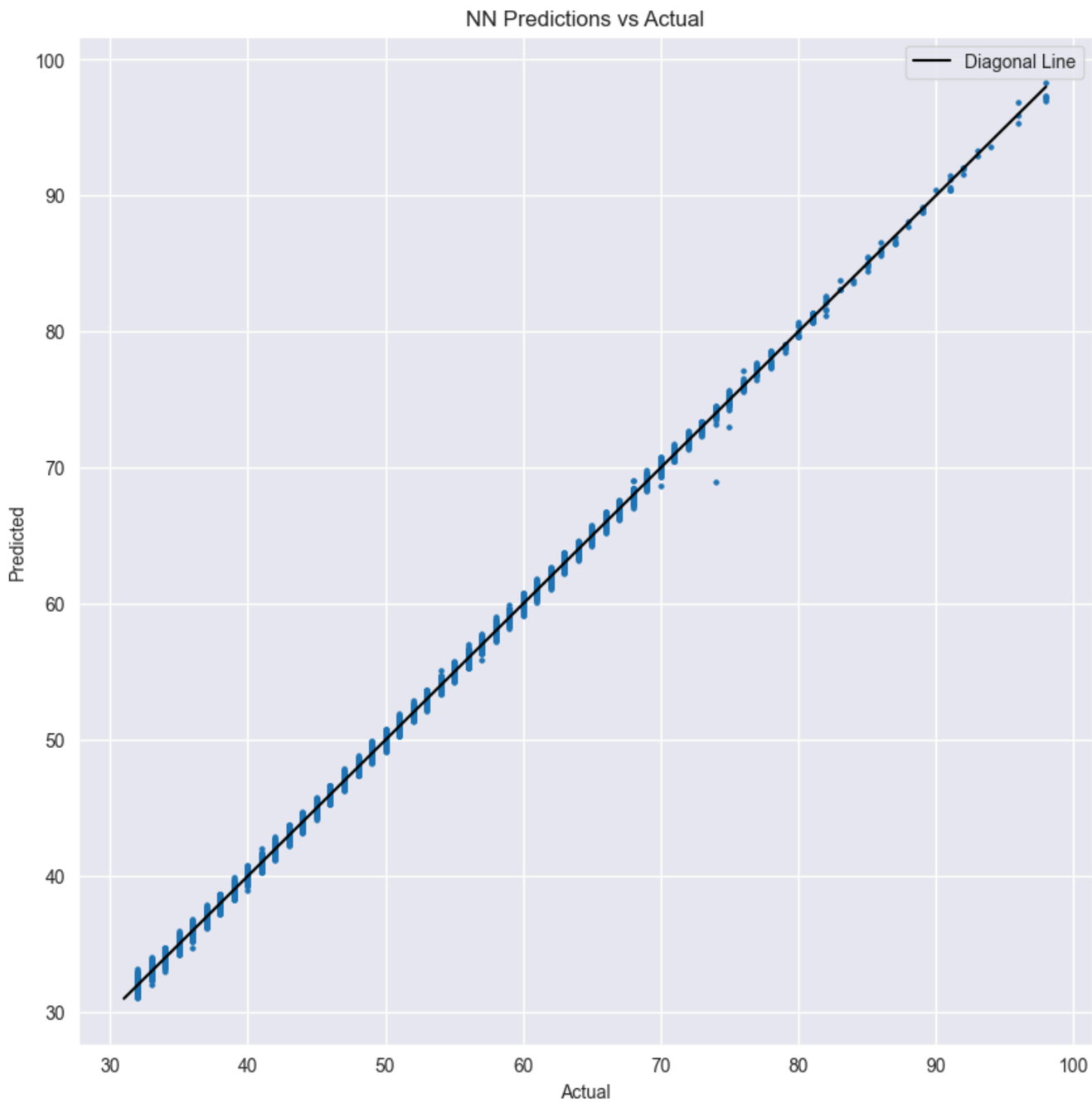
In [120…

```
def evaluate_test(model, X_test, y_test, num_cols, cat_cols, scaler):

    X_test1 = X_test.copy(deep=True)

    X_test1[num_cols] = scaler.transform(X_test1[num_cols])

    for col in cat_cols:
        categories = df[col].unique()
        mapping = {k:v for v,k in enumerate(categories)}
        cat_maps[col] = mapping
        cat_sizes[col] = int(df[col].nunique())
        X_test1[col] = X_test1[col].map(mapping).cat.codes.astype('int32')

    test_inputs = {
        "num_input": X_test1[num_cols].values
    }

    for col in cat_cols:
        test_inputs[f"{col}_input"] = X_test1[col].values

    # Keras evaluation
    loss, mae = model.evaluate(test_inputs, y_test, verbose=0)

    # Predictions
    preds = model.predict(test_inputs, verbose=0).flatten()

    # Extra metrics
    mse = np.mean((y_test - preds) ** 2)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)

    print("===== TEST METRICS =====")
    print(f"MSE  : {mse:.4f}")
    print(f"RMSE : {rmse:.4f}")
    print(f"MAE  : {mae:.4f}")
    print(f"R2   : {r2:.4f}")

    plt.figure(figsize=(10,10))
    plt.title("NN Predictions vs Actual")
    plt.scatter(y_test,
                preds,
                s=5)
    plt.plot(np.arange(y_test.min()-1,y_test.max()+1,1),
             np.arange(y_test.min()-1,y_test.max()+1,1),
             color='black',
             label='Diagonal Line')
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.legend()
    plt.show()
```

In [121…

```
evaluate_test(best_model, X_test, y_test, num_cols, cat_cols, scaler)
```

```
===== TEST METRICS =====
MSE  : 0.0978
RMSE : 0.3127
MAE  : 0.2623
R2   : 0.9989
```



Let us save our model with scaler.

In [124…

```
joblib.dump(scaler, "scaler.pkl")
best_model.save("porter_best_model.keras")
```